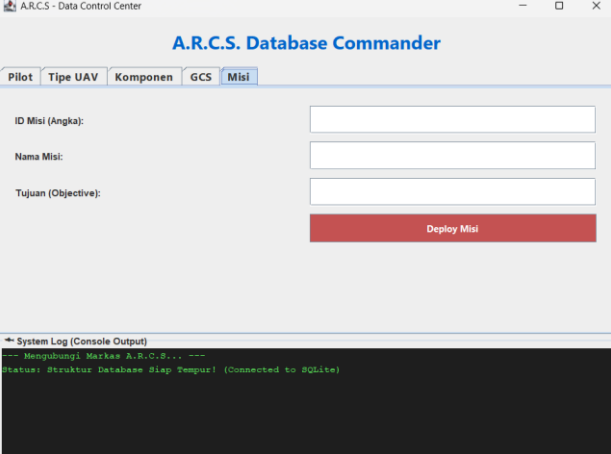
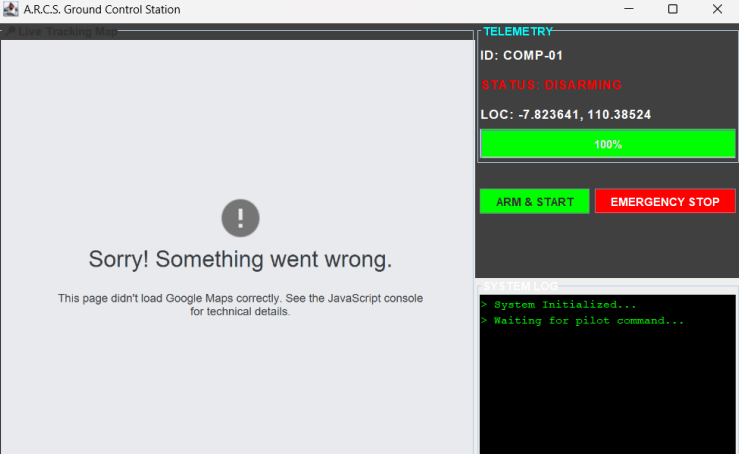


A.R.C.S. (Aerial Rescue & Color Search System)

A. Identitas proyek

- a. Identitas diri :
 - Banu Najieh Al-Fadhil (2400018060)
 - Michdan Alkahfi (2400018055)
 - Pilda Rk Pri Andini (2400018011)
- b. Judul proyek : **“PENGEMBANGAN PROTOTIPE SISTEM DETEKSI KORBAN BENCANA AIR BERBASIS PENGOLAHAN CITRA DIGITAL”.**
- c. Link : <https://github.com/OurinEdnes/A.R.C.S>
- d. Capture tampilan awal aplikasi/tampilan utama :

	
ARCS_Dashboard.java	Main.java

B. Persoalan Bisnis dan Deskripsi Proyek

Persoalan Bisnis

Pada operasi pencarian dan penyelamatan (Search and Rescue/SAR) korban bencana air, proses pemantauan masih banyak bergantung pada pengamatan visual manusia menggunakan

kamera drone. Kondisi lingkungan seperti air keruh, pencahayaan tidak stabil, area yang luas, serta kelelahan operator menyebabkan proses identifikasi korban menjadi tidak optimal dan berpotensi menimbulkan keterlambatan evakuasi.

Ketergantungan terhadap pengamatan manual juga meningkatkan risiko kesalahan deteksi korban, sehingga dibutuhkan sistem pendukung yang mampu melakukan deteksi objek secara otomatis dan real-time untuk membantu tim SAR.

Deskripsi Proyek

Proyek ini mengembangkan sebuah prototipe perangkat lunak bernama A.R.C.S. (Aerial Rescue & Color Search) berbasis Pemrograman Berorientasi Objek (PBO) menggunakan bahasa pemrograman Java dan pustaka OpenCV/JavaCV. Sistem ini memanfaatkan teknik pengolahan citra digital berbasis warna (HSV) untuk mendeteksi objek berwarna mencolok yang merepresentasikan korban atau atribut keselamatan (pelampung) pada permukaan air melalui kamera drone.

Hasil deteksi divisualisasikan dalam bentuk bounding box secara real-time untuk membantu operator dalam menentukan lokasi korban secara cepat dan akurat.

C. Daftar Seluruh Spesifikasi Aplikasi

Spesifikasi Fungsional

1. Sistem dapat menerima input video dari kamera (simulasi kamera drone).
2. Sistem mampu memecah video menjadi frame secara real-time.
3. Sistem melakukan konversi ruang warna dari RGB ke HSV.
4. Sistem mengekstraksi objek target berdasarkan rentang warna tertentu.
5. Sistem menghasilkan citra biner hasil segmentasi objek.
6. Sistem melakukan klasifikasi area objek untuk menghilangkan noise.
7. Sistem menampilkan hasil deteksi objek menggunakan bounding box.
8. Sistem berjalan secara real-time.

Spesifikasi Non-Fungsional

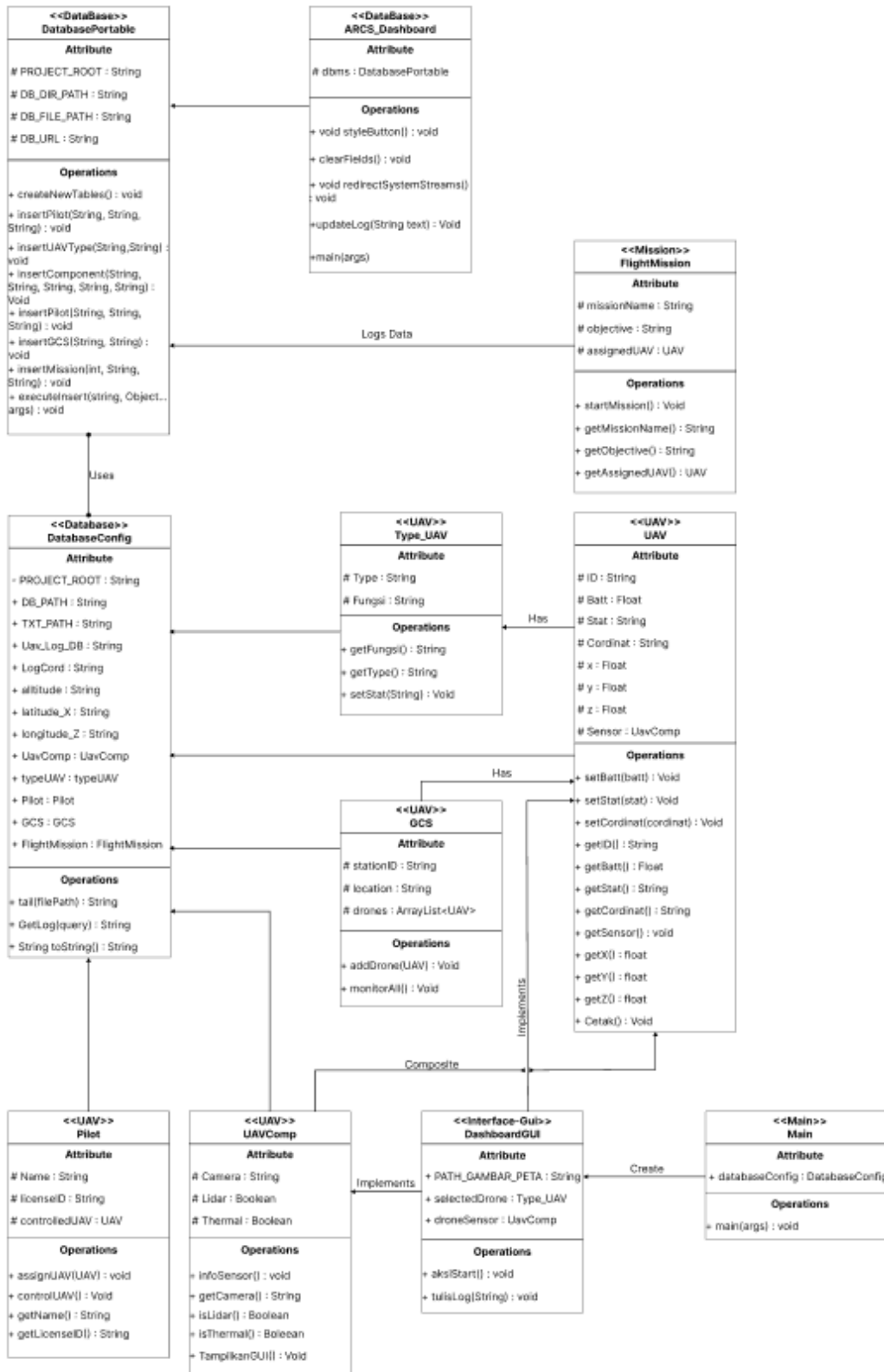
1. Bahasa pemrograman: Java.

2. Paradigma pemrograman: Object-Oriented Programming (OOP).
 3. Library pengolahan citra: OpenCV / JavaCV.
 4. Sistem berjalan pada komputer/laptop standar.
 5. Antarmuka berbasis GUI desktop.
 6. Waktu respon cepat (real-time processing).
 7. Database SQL untuk menyimpan data UAV.
-

D. Rancangan Model Diagram UML

Diagram berikut menggambarkan sebuah sistem manajemen UAV (drone) yang terintegrasi dengan database, misi penerbangan, dan dashboard pemantauan. Sistem ini terdiri dari beberapa kelas utama seperti UAV, FlightMission, DatabasePortable, dan ARCS_Dashboard yang saling berhubungan. UAV berperan sebagai objek drone yang memiliki atribut seperti ID, baterai, status, koordinat, dan sensor, serta dapat memperbarui kondisi dan posisinya. Setiap UAV memiliki tipe tertentu (Type_UAV) dan dilengkapi komponen sensor (UAVComp) seperti kamera, lidar, dan thermal. Pengendalian UAV dilakukan oleh Pilot, sedangkan pemantauan banyak UAV sekaligus dilakukan melalui GCS (Ground Control Station).

FlightMission merepresentasikan misi penerbangan yang berisi nama misi, tujuan, dan UAV yang ditugaskan untuk menjalankan misi tersebut. Seluruh data UAV, misi, dan log sistem disimpan dan dikelola oleh DatabasePortable yang berfungsi sebagai pengelola database. Konfigurasi sistem, seperti path file dan hubungan antarobjek, diatur dalam DatabaseConfig. Interaksi pengguna dilakukan melalui ARCS_Dashboard dan DashboardGUI yang berfungsi menampilkan informasi, mengelola log, serta mengontrol jalannya sistem. Keseluruhan aplikasi dijalankan melalui kelas Main yang menginisialisasi dan menghubungkan semua komponen, sehingga sistem dapat berfungsi sebagai platform pemantauan dan pengendalian UAV secara terintegrasi.



E. Rancangan Antar Muka berbasis GUI

GUI I (ARCS_Dashboard.java)

GUI I dirancang untuk mendukung pengelolaan data secara dinamis dengan mengintegrasikan database lokal SQLite melalui kelas DatabasePortable berbasis JDBC. Pemilihan SQLite memungkinkan aplikasi berjalan secara ringan, portabel, dan tanpa memerlukan konfigurasi server. Sistem memastikan keamanan dan integritas data dengan menggunakan Prepared Statement serta secara otomatis memverifikasi dan menyiapkan tabel data untuk entitas Pilot, UAV, dan Misi setiap kali aplikasi dijalankan, sehingga proses input dan pengelolaan data dapat dilakukan dengan aman dan stabil.

GUI II (Main.java)

GUI II berfungsi sebagai Ground Control Station (GCS) yang menampilkan visualisasi dan pemantauan UAV secara real-time melalui peta interaktif. Antarmuka ini menggabungkan Java Swing dan JavaFX untuk menampilkan Live Tracking Map berbasis Google Maps API, sehingga posisi UAV dapat dipantau secara akurat. Untuk menjaga responsivitas sistem, proses akuisisi data sensor dijalankan secara multithreading, sementara panel telemetry menampilkan informasi penting seperti status baterai, koordinat, dan ID UAV secara dinamis guna mendukung kesadaran situasional operator.

F . Skrip Program dan penjelasannya

- Class UavComp

Class ini merupakan implementasi sistem pengolahan citra digital (*digital image processing*) yang dirancang untuk kebutuhan pemantauan pada UAV (Unmanned Aerial Vehicle). Sistem ini difokuskan pada deteksi hipotesis korban melalui identifikasi warna spesifik pada aliran video langsung (*real-time stream*).

Poin-poin utama penjelasan:

1. Integrasi Perangkat Keras (Stream Grabbing):

Sistem menggunakan pustaka JavaCV dan FFmpegFrameGrabber untuk menangkap aliran video baik melalui alamat IP (MJPEG) maupun input kamera lokal. Terdapat fungsi *fallback* otomatis ke kamera standar (ID 0) jika alamat kamera yang ditentukan tidak ditemukan.

2. Konversi Ruang Warna (Color Space Conversion):

Proses ekstraksi fitur dimulai dengan mengubah ruang warna dari BGR (Blue, Green, Red) menjadi HSV (Hue, Saturation, Value). Penggunaan HSV lebih efektif dibandingkan BGR dalam deteksi objek karena memisahkan informasi warna (Hue) dari intensitas cahaya, sehingga lebih stabil terhadap perubahan pencahayaan di luar ruangan.

3. Segmentasi Warna (Thresholding):

Sistem menerapkan teknik *thresholding* menggunakan fungsi `inRange` untuk mengisolasi rentang warna merah (yang diidentifikasi sebagai penanda target atau korban). Karena spektrum warna merah berada di dua ujung nilai Hue pada OpenCV (0-10 dan 160-179), sistem melakukan dua kali mask kemudian menggabungkannya menggunakan fungsi `addWeighted`.

4. Analisis Kontur dan Filter Area:

Setelah mendapatkan masker biner, sistem melakukan pencarian kontur (*contour detection*). Untuk meminimalisir *noise* (gangguan visual), sistem menerapkan filter luas area; hanya objek dengan luas lebih dari 500 piksel yang akan diproses dan diberi label.

5. Visualisasi Data:

Output sistem berupa dua jendela visual:

- a. UAV-CAM_View: Menampilkan video asli dengan kotak pembatas (*bounding box*) hijau dan teks "Hipotesis Korban" pada objek yang terdeteksi.
- b. UAV-CAM-Mask-Threshold: Menampilkan hasil segmentasi biner untuk keperluan analisis teknis ambang batas warna.

```

12 public class UavComp implements CamView{ 7 usages & Decadanza +1
73 public void TampilkanGUI() throws FrameGrabber.Exception {
75     grabber.start();
76     while((frame = grabber.grab()) != null){
77         img = converter.convert(frame);
78         if (img == null) continue;
79
80         Mat hsv = new Mat();
81         cvtColor(img,hsv,COLOR_BGR2HSV);
82
83         Scalar lowerRed = new Scalar(0, 100, 100, 0);
84         Scalar upperRed = new Scalar(10, 255, 255, 0);
85         Scalar lowerRed2 = new Scalar(160, 100, 100, 0);
86         Scalar upperRed2 = new Scalar(179, 255, 255, 0);
87
88         Scalar lowerYellow = new Scalar(20, 100, 100, 0);
89         Scalar upperYellow = new Scalar(35, 255, 255, 0);
90
91         Mat mask1 = new Mat();
92         Mat mask2 = new Mat();
93         Mat mask = new Mat();
94
95         inRange(hsv, new Mat(lowerRed), new Mat(upperRed), mask1);
96         inRange(hsv, new Mat(lowerRed2), new Mat(upperRed2), mask2);
97         addWeighted(mask1, v: 1.0, mask2, v1: 1.0, v2: 0.0, mask);
98
99         MatVector contours = new MatVector();
100         findContours(mask, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
101
102         for (long i = 0; i < contours.size(); i++) {
103             Mat contour = contours.get(i);
104
105             double area = contourArea(contour);
106
107             if (area > 500) {
108
109                 Rect rect = boundingRect(contour);
110
111                 rectangle(img, rect, new Scalar(0, 255, 0, 0), t: 2, LINE_8, i2: 0);
112                 rectangle(mask, rect, new Scalar(255, 255, 255, 0), t: 2, LINE_8, i2: 0);
113
114                 putText(img, s: "Hipotesis Korban", new Point(rect.x(), _y: rect.y() - 10),
115                     FONT_HERSHEY_PLAIN, v: 0.7, new Scalar(0, 0, 0, 0), it: 1, LINE_AA, b: false);
116                 putText(mask, s: "Hipotesis Korban", new Point(rect.x(), _y: rect.y() - 10),
117                     FONT_HERSHEY_PLAIN, v: 0.7, new Scalar(255, 255, 255, 0), it: 1, LINE_AA, b: false);

```

- Class UAV

Class UAV ini dirancang sebagai representasi objek pesawat nirawak menggunakan paradigma Pemrograman Berorientasi Objek (PBO).

Seluruh atribut (ID, batt, Stat, Cordinat, x, y, z) menggunakan *modifier* protected. Hal ini bertujuan untuk menyembunyikan data dari akses publik langsung, namun tetap memungkinkan akses bagi kelas turunan (*subclass*) dalam skema pewarisan (*inheritance*).

Atribut sensor yang bertipe `UavComp` merepresentasikan hubungan *Has-A*. Ini menunjukkan bahwa setiap objek UAV memiliki komponen sensor yang terpisah namun menjadi bagian integral dari sistem utamanya.

Konstruktor tidak hanya menginisialisasi variabel, tetapi juga melakukan pemrosesan data (*data processing*). String Cordinat dipecah menggunakan metode `.split(",")` dan dikonversi (*type casting*) menjadi float untuk mengisi nilai sumbu spasial (x, y, z) secara otomatis.

Metode Akses dan Mutasi:

- Setter/Getter: Digunakan untuk memanipulasi dan mengambil data atribut dengan aman. Pada `setCordinat`, logika pemecahan string kembali diterapkan untuk memastikan sinkronisasi antara data string dan koordinat numerik.
- Delegasi & Output: Metode `getSensor()` mendelegasikan tugas informasi ke objek sensor, sedangkan `Cetak()` berfungsi menampilkan ringkasan status operasional (ID, Baterai, Status) ke konsol.


```

1 public class UAV {  ⚡ Decadanza +1
2     protected String ID; 3 usages
3     protected float batt; 4 usages
4     protected String Stat; 4 usages
5     protected String Cordinat; 3 usages
6     protected float x; 3 usages
7     protected float y; 2 usages
8     protected float z; 3 usages
9     protected UavComp sensor; // Composite 2 usages
10
11 @ public UAV(String cordinat, String stat, float batt, String ID, UavComp sensor) {  ⚡ Decadanza +1
12     this.Cordinat = cordinat;
13     this.Stat = stat;
14     this.batt = batt;
15     this.ID = ID;
16     this.sensor = sensor;
17
18     String[] part = cordinat.split(regex: " "); // pisah koordinat by koma
19
20     this.x = Float.parseFloat(part[0]);
21     this.z = Float.parseFloat(part[1]);
22 }
23
24 // ==== Setter ====
25 > public void setBatt(float batt) { this.batt = batt; }
26
27
28
29 ⚡ > public void setStat(String stat) { Stat = stat; }
30
31
32
33 @ public void setCordinat(String cordinat) { no usages  ⚡ Decadanza
34     Cordinat = cordinat;
35
36     String[] part = cordinat.split(regex: " "); // pisah koordinat by koma
37
38     this.x = Float.parseFloat(part[0]);
39     this.y = Float.parseFloat(part[1]);
40     this.z = Float.parseFloat(part[2]);
41 }

```

- Class Type_UAV

Class Type_UAV merupakan kelas turunan dari kelas UAV yang dirancang untuk menambahkan informasi khusus terkait jenis dan fungsi wahana udara tanpa awak. Dengan menggunakan konsep *inheritance*, kelas ini mewarisi seluruh atribut dan metode dari kelas UAV, kemudian memperluasnya dengan atribut tambahan berupa type dan fungsi. Proses inisialisasi objek dilakukan melalui konstruktor yang tidak hanya memanggil konstruktor kelas induk untuk mengatur data dasar UAV, tetapi juga menentukan jenis UAV berdasarkan parameter tertentu. Jenis UAV dibedakan menjadi beberapa kategori, seperti *Fixed Wing Long Range*, *V-TOL*, dan *Lela*, sedangkan nilai di luar ketentuan akan diklasifikasikan sebagai UAV yang tidak dikenal. Apabila jenis UAV valid, maka fungsi utama UAV ditetapkan sebagai operasi penyelamatan darurat. Kelas ini juga menerapkan prinsip *encapsulation* melalui penggunaan metode *getter* untuk mengakses atribut, serta *method overriding* untuk mempertahankan perilaku metode yang diwarisi dari kelas induk. Dengan demikian, class Type_UAV berperan

sebagai pengembangan dari class UAV yang lebih spesifik dan terstruktur sesuai kebutuhan sistem.

```
1 public class Type_UAV extends UAV { 4 usages 2 Decadanza +1
2     protected String type; 5 usages
3     protected String fungsi; 3 usages
4
5     public Type_UAV(String coordinate, String stat, float batt, String ID, int a, UavComp sensor) { 1 usage 2 Decadanza +1
6         super(coordinate, stat, batt, ID, sensor);
7
8         if (a == 1) {
9             this.type = "Fixed Wing Long Range";
10        } else if (a == 2) {
11            this.type = "V-TOL";
12        } else if (a == 3) {
13            this.type = "Lela";
14        } else {
15            this.type = "UnknownDrone";
16            this.fungsi = "Tidak diketahui";
17            return;
18        }
19        this.fungsi = "Operasi penyelamatan darurat";
20    }
21
22
23    > public String getFungsi() { return fungsi; }
24
25
26    > public String getType() { return type; }
27
28
29
30
31    @Override 2 usages 2 Decadanza
32    > public void setStat(String stat) { super.setStat(stat); }
33
34
35
36    @Override 1 usage 2 Decadanza
37    > public void Cetak() { super.Cetak(); }
38
39
40 }
```

- Class pilot

Class Pilot merupakan kelas yang merepresentasikan seorang operator atau pilot yang bertugas mengendalikan wahana udara tanpa awak (UAV). Kelas ini memiliki atribut berupa nama dan nomor lisensi pilot sebagai identitas, serta sebuah referensi ke objek UAV yang dikendalikan, yang menunjukkan adanya hubungan **asosiasi** antara pilot dan UAV. Melalui konstruktor, data identitas pilot diinisialisasi saat objek dibuat. Kelas ini menyediakan mekanisme untuk menghubungkan seorang pilot dengan UAV tertentu, sehingga pilot dapat ditugaskan untuk mengoperasikan drone tersebut. Selain itu, terdapat metode yang memungkinkan pilot menampilkan informasi penting terkait UAV yang sedang dikendalikan, seperti identitas UAV, status, dan tingkat baterai. Apabila pilot belum ditugaskan pada UAV mana pun, sistem akan memberikan informasi bahwa tidak ada UAV yang dikendalikan. Penerapan metode *getter* pada kelas ini mendukung prinsip *encapsulation*, sehingga data pilot tetap terlindungi namun tetap dapat diakses secara terkontrol oleh bagian lain dari sistem.

```

1 public class Pilot { 2 usages 2 Decadanza
2     private String name; 5 usages
3     private String licenseID; 2 usages
4     private UAV controlledUAV; // Association 5 usages
5
6     public Pilot(String name, String licenseID) { 1 usage 2 Decadanza
7         this.name = name;
8         this.licenseID = licenseID;
9     }
10
11     // Connect Pilot to UAV
12 @ public void assignUAV(UAV drone) { no usages 2 Decadanza
13     this.controlledUAV = drone;
14     System.out.println("Pilot " + name + " assigned to UAV " + drone.getID());
15 }
16
17 public void controlUAV() { no usages 2 Decadanza
18     if (controlledUAV != null) {
19         System.out.println("\n=== UAV Control Panel ===");
20         System.out.println("Pilot : " + name);
21         System.out.println("Operating UAV : " + controlledUAV.getID());
22         System.out.println("Battery : " + controlledUAV.getBatt());
23         System.out.println("Status : " + controlledUAV.getStat());
24         System.out.println("=====");
25     } else {
26         System.out.println("Pilot " + name + " has no UAV assigned!");
27     }
28 }
29
30 // Getter
31 public String getName() { return name; } no usages 2 Decadanza
32 public String getLicenseID() { return licenseID; } no usages 2 Decadanza
33 }

```

- Class GCS

Class GCS (Ground Control Station) merupakan kelas yang merepresentasikan pusat kendali darat yang berfungsi untuk memantau dan mengelola beberapa UAV sekaligus. Kelas ini memiliki atribut berupa stationID sebagai identitas stasiun, location sebagai lokasi operasional, serta sebuah kumpulan objek UAV yang disimpan dalam struktur ArrayList. Hubungan antara GCS dan UAV bersifat aggregation, yang berarti GCS hanya berperan sebagai pengelola atau pemantau UAV, namun tidak memiliki kepemilikan penuh terhadap UAV tersebut—UAV tetap dapat berdiri secara independen tanpa GCS. Melalui konstruktor, identitas dan lokasi GCS diinisialisasi, serta daftar UAV disiapkan sebagai wadah penyimpanan. Kelas ini menyediakan mekanisme untuk menambahkan UAV ke dalam sistem pemantauan GCS dan menampilkan informasi bahwa UAV telah terhubung. Selain itu, GCS memiliki kemampuan untuk memantau seluruh UAV yang terhubung dengan menampilkan informasi penting seperti jumlah UAV, identitas, status baterai, kondisi operasional, dan posisi masing-masing UAV. Dengan demikian, class GCS berperan sebagai

komponen sentral dalam sistem yang memastikan proses pemantauan UAV dapat dilakukan secara terstruktur dan terpusat.

```
1  import java.util.ArrayList;
2
3  public class GCS { 2 usages  ⚙ Decadanza
4      private String stationID; 2 usages
5      private String location; 2 usages
6      private ArrayList<UAV> drones; // AGGREGATION: GCS tidak memiliki UAV sepenuhnya 4 usages
7
8      public GCS(String stationID, String location) { 1 usage  ⚙ Decadanza
9          this.stationID = stationID;
10         this.location = location;
11         this.drones = new ArrayList<>();
12     }
13
14     public void addDrone(UAV drone) { no usages  ⚙ Decadanza
15         drones.add(drone);
16         System.out.println("UAV " + drone.getID() + " connected to GCS.");
17     }
18
19     public void monitorAll() { no usages  ⚙ Decadanza
20         System.out.println("\n=== Ground Control Monitoring (" + stationID + ") ===");
21         System.out.println("Location: " + location);
22         System.out.println("Total Connected UAV: " + drones.size());
23         System.out.println("-----");
24
25         for (UAV d : drones) {
26             System.out.println("ID      : " + d.getID());
27             System.out.println("Battery : " + d.getBatt() + "%");
28             System.out.println("Status  : " + d.getStat());
29             System.out.println("Position : " + d.getCordinat());
30             System.out.println("-----");
31         }
32     }
33 }
```

- Class FlightMission

Class FlightMission merupakan kelas yang digunakan untuk merepresentasikan sebuah misi penerbangan yang dijalankan oleh UAV. Kelas ini menyimpan informasi penting terkait misi, seperti nama misi dan tujuan misi, serta memiliki referensi ke satu objek UAV yang ditugaskan untuk melaksanakan misi tersebut. Hubungan antara FlightMission dan UAV bersifat association, karena kelas ini hanya menyimpan referensi UAV tanpa memiliki atau mengelola siklus hidupnya. Melalui konstruktor, seluruh informasi misi dan UAV yang ditugaskan diinisialisasi sejak awal pembuatan objek. Kelas ini menyediakan fungsi untuk memulai misi dengan menampilkan detail misi beserta status UAV yang sedang menjalankannya. Selain itu, metode *getter* disediakan untuk memungkinkan akses terkontrol terhadap data misi dan UAV sesuai dengan prinsip *encapsulation*. Dengan demikian, class FlightMission berperan sebagai pengelola informasi operasional misi yang terhubung langsung dengan UAV dalam sistem.

```

1 public class FlightMission { 2 usages  ⚡ Decadanza
2     private String missionName; 3 usages
3     private String objective; 3 usages
4     private UAV assignedUAV; // Association, hanya reference 4 usages
5
6     public FlightMission(String missionName, String objective, UAV assignedUAV) { 1 usage  ⚡ Decadanza
7         this.missionName = missionName;
8         this.objective = objective;
9         this.assignedUAV = assignedUAV; // UAV terhubung tapi tidak dimiliki
10    }
11
12    public void startMission() { no usages  ⚡ Decadanza
13        System.out.println("\n== Mission Activated ==");
14        System.out.println("Mission : " + missionName);
15        System.out.println("Objective : " + objective);
16        System.out.println("Assigned UAV : " + assignedUAV.getID());
17        System.out.println("Status UAV : " + assignedUAV.getStat());
18        System.out.println("=====");
19    }
20
21    // Getter optional
22    public String getMissionName() { return missionName; } no usages  ⚡ Decadanza
23    public String getObjective() { return objective; } no usages  ⚡ Decadanza
24    public UAV getAssignedUAV() { return assignedUAV; } no usages  ⚡ Decadanza
25 }

```

- Class DatabaseConfig

Class DatabaseConfig berfungsi sebagai komponen konfigurasi dan inisialisasi data utama dalam sistem A.R.C.S yang mengintegrasikan sumber data file log teks dan database SQLite. Kelas ini bertanggung jawab untuk mendeteksi lokasi root proyek secara otomatis agar sistem dapat berjalan lintas platform (Windows maupun Linux), kemudian menentukan path database dan file log secara dinamis. Pada saat objek dibuat, kelas ini membaca data dinamis UAV seperti latitude, longitude, dan ketinggian dari baris terakhir file log teks, sehingga informasi posisi UAV selalu bersifat terbaru. Selain itu, DatabaseConfig juga mengambil data statis dari database SQLite untuk membangun objek-objek inti sistem, seperti komponen UAV, tipe UAV, pilot, Ground Control Station (GCS), dan misi penerbangan, yang saling terhubung melalui relasi asosiasi dan agregasi. Dengan pendekatan ini, kelas DatabaseConfig berperan sebagai pusat pemuatan data (data loader) yang menyatukan berbagai sumber informasi menjadi satu konfigurasi sistem yang siap digunakan. Adanya metode pendukung untuk membaca baris terakhir file dan mengeksekusi query database memastikan proses pengambilan data berjalan terstruktur dan efisien, sedangkan metode toString() digunakan untuk menampilkan ringkasan status sistem secara informatif.

```

37 public DatabaseConfig() throws FileReaderException { Usage & Decadanza *
38     System.out.println("Loading Konfigurasi A.R.C.S...");
39     System.out.println("Deteksi Root Project: " + PROJECT_ROOT);
40     System.out.println("Target Database: " + DB_PATH);
41
42     // 1. AMBIL DATA DINAMIS (Dari File TXT via jurus Tail)
43     String lastLine = tail(LogCord);
44     if (lastLine != null) {
45         try {
46             String[] temp = lastLine.split(" ");
47             // [0]Type, [1]Date, [2]Lat, [3]Lon, [4]Acc, [5]Alt
48             this.latitude_X = temp[2];
49             this.longitude_Z = temp[3];
50             this.altitude = temp[5] + " mdp1";
51         } catch (Exception e) {
52             System.out.println(" Error parsing TXT: " + e.getMessage());
53         }
54     } else {
55         System.out.println("Warning: File Log TXT tidak ditemukan di path: " + LogCord);
56     }
57
58     // == UavComp ==
59     String dataComp = GetLog("query: *SELECT * FROM UAV_Components LIMIT 1");
60     if (dataComp != null) {
61         String[] c = dataComp.split(" ");
62         boolean hasLidar = (c.length > 2 && !c[2].equalsIgnoreCase("null") && !c[2].isEmpty());
63         boolean hasThermal = (c.length > 3 && !c[3].equalsIgnoreCase("null") && !c[3].isEmpty());
64
65         this.UavComp = new UavComp(c[1], hasLidar, hasThermal, alarmCam: 0);
66     }
67
68     // == Type_UAV ==
69     String dataType = GetLog("query: *SELECT * FROM Type_UAV LIMIT 1");
70     if (dataType != null) {
71         String[] t = dataType.split(" ");
72         int kmit = 0;
73         try {
74             kmit = Integer.parseInt(t[1]);
75         } catch (NumberFormatException e) {
76             // Handle error parsing
77         }
78         this.typeUAV = new Type_UAV("coordinate: this.latitude_X + "," + this.longitude_Z + "," + this.altitude, stat: "DISARMING", batt: 100.0f, t[0], kmit, this.UavComp);
79     }

```

- Class DashboardGUI

Class DashboardGUI merupakan kelas antarmuka grafis (*Graphical User Interface*) yang berfungsi sebagai Ground Control Station visual dalam sistem A.R.C.S. Kelas ini dibangun menggunakan Java Swing yang dikombinasikan dengan JavaFX, sehingga mampu menampilkan informasi telemetry UAV secara real-time sekaligus peta pelacakan berbasis Google Maps. DashboardGUI menampilkan data penting UAV seperti identitas drone, status operasional, posisi koordinat, dan level baterai dalam bentuk visual yang mudah dipahami. Selain itu, kelas ini menyediakan kontrol interaktif berupa tombol untuk memulai operasi UAV dan menampilkan log sistem sebagai umpan balik aktivitas pengguna. Kelas ini juga terhubung langsung dengan objek Type_UAV dan UavComp, yang memungkinkan dashboard tidak hanya bersifat informatif, tetapi juga interaktif dalam mengendalikan status UAV dan mengaktifkan sensor kamera. Dengan desain ini, DashboardGUI berperan sebagai pusat kendali visual yang mengintegrasikan data, kontrol, dan pemantauan UAV dalam satu tampilan terpadu yang mendukung pengoperasian sistem secara efektif dan responsif.

```

50 public DashboardGUI(Type_UAV drone, UavComp sensor) { Usage & Decadanza
51     this.selectedDrone = drone;
52     this.droneSensor = sensor;
53
54     // 1. Setup Jendela Utama (Frame)
55     setTitle("A.R.C.S. Ground Control Station");
56     setSize( width: 800, height: 500);
57     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
58     setLayout(new BorderLayout());
59
60     getContentPane().setBackground(Color.DARK_GRAY);
61
62     JFXPanel panelPeta = new JFXPanel();
63     panelPeta.setBorder(BorderFactory.createTitledBorder("📍 Live Tracking Map"));
64     panelPeta.setPreferredSize(new Dimension( width: 500, height: 500));
65
66     add(panelPeta, BorderLayout.CENTER);
67
68     String htmlMap = "<html>"
69         + "<head>"
70         + "    <title>UAV Tracking</title>"
71         + "    <style>"
72         + "        html, body, gmp-map { height: 100%; margin: 0; padding: 0; }"
73         + "    </style>"
74         + "    // Script Google Maps (Jangan lupa API KEY-nya ya Senpai!)"
75         + "    <script src='https://maps.googleapis.com/maps/api/js?key=AIzaSyA6mVHzS10YXdcazAFaImXv0KcYcP5cLc8&libraries=maps,marker&v=weekly' defer></script>"
76         + "</head>"
77         + "<body>"
78         + "    <gmp-map center='-7.82364089, 110.38523845' zoom='14' map-id='DEMO_MAP_ID'>"
79         + "        <gmp-advanced-marker position='-7.82364089, 110.38523845' title='Posisi UAV'></gmp-advanced-marker>"
80         + "    </gmp-map>"
81         + "</body>"
82         + "</html>";
83
84     Platform.runLater(() -> {
85         WebView webView = new WebView();
86         // Load HTML String tadi ke dalam browser mini ini
87         webView.getEngine().loadContent(htmlMap);
88
89         // Masukan browsernya ke panel
90         panelPeta.setScene(new Scene(webView));
91     });

```

- Class DatabasePortable

Class DatabasePortable merupakan kelas yang dirancang untuk menangani pembuatan dan pengelolaan database SQLite secara portable dalam sistem A.R.C.S. Kelas ini secara otomatis mendeteksi direktori root proyek agar lokasi database dapat disesuaikan dengan lingkungan eksekusi tanpa bergantung pada path statis, sehingga aplikasi dapat dijalankan di berbagai sistem operasi. DatabasePortable bertanggung jawab untuk memastikan direktori penyimpanan database tersedia, membuat struktur tabel yang dibutuhkan sistem (seperti data UAV, komponen, pilot, GCS, dan misi), serta mengelola proses pengisian data awal ke dalam database. Selain itu, kelas ini menyediakan mekanisme untuk memasukkan dan memperbarui data dengan pendekatan *auto-reset*, di mana data lama dihapus sebelum data baru dimasukkan agar konsistensi sistem tetap terjaga. Fungsi tambahan juga disediakan untuk menampilkan isi tabel sebagai sarana verifikasi dan debugging. Dengan perannya tersebut, class DatabasePortable berfungsi sebagai fondasi manajemen basis data yang memastikan seluruh komponen sistem A.R.C.S memiliki sumber data yang terstruktur, konsisten, dan siap digunakan.

```

64 public void createNewTables() { //usage : Decadanza
65     // Aiko Check: Bikin folder dulu kalau belum ada!
66     File directory = new File(DB_DIR_PATH);
67     if (!directory.exists()) {
68         if (directory.mkdirs()) {
69             System.out.println("Magic: Folder 'UAV-LOG' berhasil dibuat otomatis!");
70         } else {
71             System.out.println("Warning: Gagal bikin folder, mungkin perlu izin admin/root.");
72         }
73     }
74
75     // Mengubah CamAdd dari INT ke TEXT agar bisa menyimpan URL
76     String[] sqls = {
77         "CREATE TABLE IF NOT EXISTS Type_UAV (id CHAR(6) PRIMARY KEY, nama TEXT);",
78         "CREATE TABLE IF NOT EXISTS UAV_Components (id CHAR(6) PRIMARY KEY, Cam TEXT, Lidar TEXT, Thermal TEXT, CamAdd TEXT);",
79         "CREATE TABLE IF NOT EXISTS Pilot (id CHAR(6) PRIMARY KEY, Nama TEXT, LicenseID TEXT);",
80         "CREATE TABLE IF NOT EXISTS GCS (id CHAR(6) PRIMARY KEY, Location TEXT);",
81         "CREATE TABLE IF NOT EXISTS Mission_Task (id INTEGER PRIMARY KEY, MissionName TEXT, Objective TEXT);"
82     };
83
84     // Load Driver Explicitly (Just in case)
85     try { Class.forName("org.sqlite.JDBC"); } catch (Exception e) {}
86
87     try (Connection conn = DriverManager.getConnection(DB_URL);
88         Statement stmt = conn.createStatement()) {
89
90         for (String sql : sqls) {
91             stmt.execute(sql);
92         }
93
94         System.out.println("Status: Struktur Database Siap Tempun! (Connected to SQLite)");
95
96     } catch (SQLException e) {
97         System.out.println("☹ Gagal bikin tabel: " + e.getMessage());
98         System.out.println("Cek apakah folder 'UAV-LOG' bisa diakses?");
99     }
100 }

```

- Class ARCS_Dashboard

Class ARCS_Dashboard merupakan kelas antarmuka grafis yang berfungsi sebagai pusat kendali dan manajemen data dalam sistem A.R.C.S berbasis desktop. Kelas ini dibangun menggunakan Java Swing untuk menyediakan tampilan interaktif yang memungkinkan pengguna mengelola seluruh data penting sistem, seperti pilot, tipe UAV, komponen UAV, Ground Control Station (GCS), dan misi penerbangan, melalui satu dashboard terintegrasi. ARCS_Dashboard memanfaatkan mekanisme *tabbed pane* agar setiap kategori data ditampilkan dalam tab terpisah sehingga antarmuka tetap rapi dan mudah digunakan. Kelas ini juga terhubung langsung dengan kelas DatabasePortable sebagai pengelola basis data, sehingga setiap input dari pengguna dapat langsung disimpan dan diperbarui ke dalam database SQLite. Selain itu, dashboard ini dilengkapi dengan area log yang secara otomatis menampilkan keluaran sistem dengan mengalihkan *console output* ke dalam komponen GUI, sehingga proses eksekusi dan status sistem dapat dipantau secara real time. Dengan peran tersebut, class ARCS_Dashboard bertindak sebagai *control center* yang mempermudah pengelolaan data dan pemantauan sistem A.R.C.S secara terstruktur, interaktif, dan terpusat.

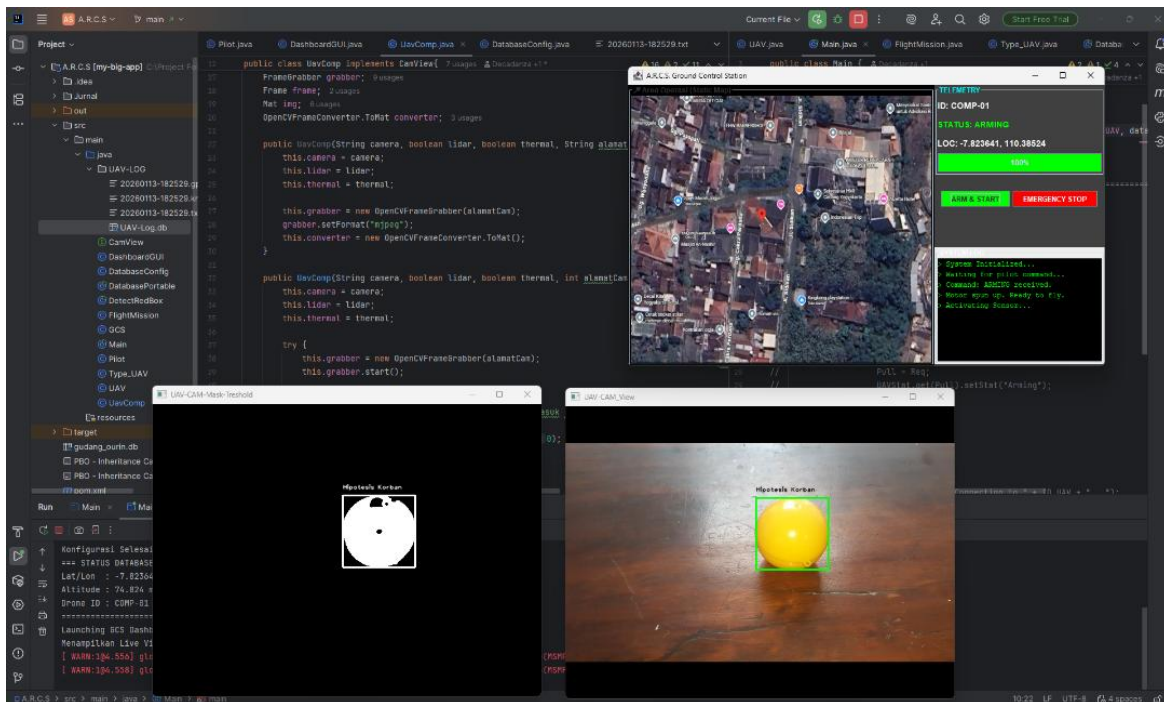

```

9 public class ARCS_Dashboard extends JFrame { & Decadanza
12     private DatabasePortable dbms; 7 usages
13
14     // Komponen UI
15     private JTabbedPane tabbedPane; 8 usages
16     private JTextArea logArea; 9 usages
17
18     public ARCS_Dashboard() { 1 usage & Decadanza
19         // 1. Setup Jendela Utama (The Frame)
20         setTitle("A.R.C.S - Data Control Center");
21         setSize(width: 800, height: 600); // Aiko gedein dikit biar lega
22         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23         setLocationRelativeTo(null); // Biar muncul di tengah layar
24         setLayout(new BorderLayout());
25
26         // Header Keren a la Aiko
27         JLabel titleLabel = new JLabel(text: "A.R.C.S. Database Commander", SwingConstants.CENTER);
28         titleLabel.setFont(new Font(name: "Segoe UI", Font.BOLD, size: 24));
29         titleLabel.setForeground(new Color(r: 0, g: 102, b: 204)); // Warna Biru Laut
30         titleLabel.setBorder(BorderFactory.createEmptyBorder(top: 15, left: 10, bottom: 15, right: 10));
31         add(titleLabel, BorderLayout.NORTH);
32
33         // 2. Area Log (Disiapin duluan biar bisa nangkap log inialisasi)
34         logArea = new JTextArea(rows: 8, columns: 20);
35         logArea.setEditable(false);
36         logArea.setFont(new Font(name: "Monospaced", Font.PLAIN, size: 12));
37         logArea.setBackground(new Color(r: 30, g: 30, b: 30)); // Dark mode log
38         logArea.setForeground(new Color(r: 100, g: 255, b: 100)); // Hacker green text
39         JScrollPane scrollLog = new JScrollPane(logArea);
40         scrollLog.setBorder(BorderFactory.createTitledBorder("🔊 System Log (Console Output)"));
41         add(scrollLog, BorderLayout.SOUTH);
42
43         // 3. JURUS RAHASIA: Redirect System.out ke JTextArea
44         // Jadi apapun yang di-print sama DatabasePortable bakal muncul disini!
45         redirectSystemStreams();
46
47         // 4. Inisialisasi Database (Panggil Class Sebelah)
48         System.out.println("--- Mengubungi Markas A.R.C.S... ---");
49         try {
50             this.dbms = new DatabasePortable();
51             this.dbms.createNewTables(); // Bikin tabel kalau belum ada
52         } catch (Exception e) {
53             System.out.println("ERROR FATAL: " + e.getMessage());
54         }

```

G. Penjelasan screenshot tampilan yang dihasilkan aplikasi

Berdasarkan hasil eksperimen, sistem menunjukkan performa yang optimal dalam mendeteksi objek bola kuning pada kondisi pencahayaan ideal. Sebagaimana diperlihatkan pada Gambar 4, sistem berhasil memisahkan area objek dari latar belakang melalui proses *thresholding*, yang menghasilkan citra biner yang bersih. Koordinat objek yang terdeteksi kemudian divisualisasikan secara akurat menggunakan *bounding box* pada *frame* asli.



Pengujian sistem deteksi objek target berbasis pengolahan citra digital dilakukan menggunakan data video simulasi yang diperoleh dari perekaman kamera ponsel pada lingkungan terkontrol. Objek bola berwarna kuning digunakan sebagai representasi visual atribut keselamatan berupa *lifebuoy* untuk mensimulasikan skenario pemantauan oleh wahana udara tanpa awak (*Unmanned Aerial Vehicle / UAV*). Sistem deteksi dikembangkan menggunakan bahasa pemrograman Java dengan pendekatan Pemrograman Berorientasi Objek (PBO), serta mengintegrasikan pustaka OpenCV sebagai *computer vision engine* untuk mendukung pemrosesan citra dan video secara real-time.

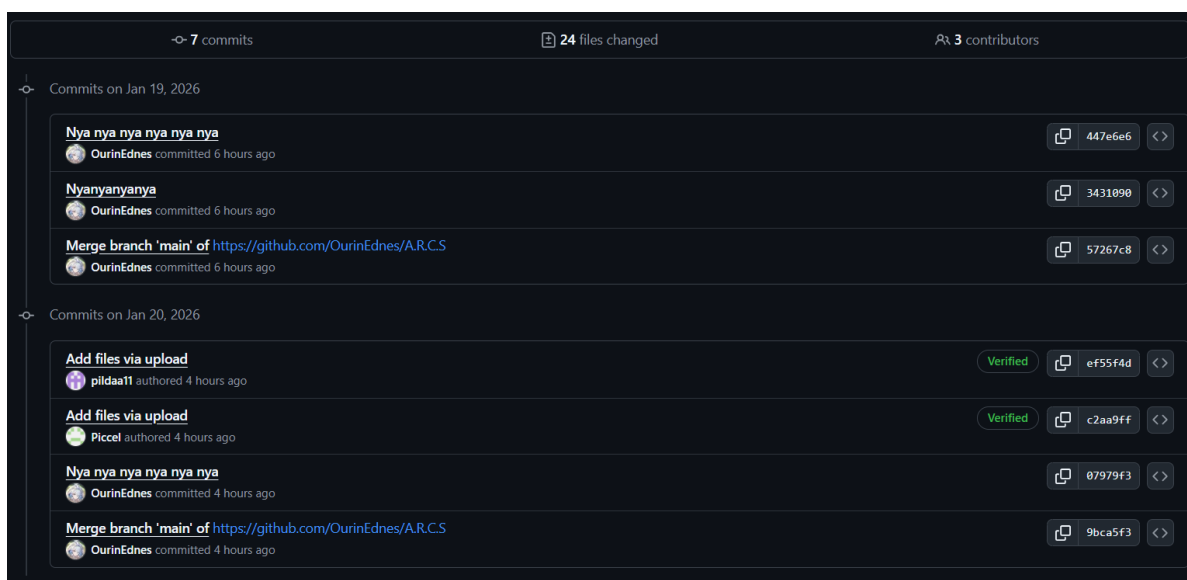
Validasi sistem dilaksanakan berdasarkan tahapan pemrosesan citra yang telah dirancang, dimulai dari konversi ruang warna RGB ke HSV guna meningkatkan ketahanan terhadap variasi pencahayaan, dilanjutkan dengan proses *thresholding* untuk mengekstraksi fitur warna objek target. Selanjutnya, operasi morfologi diterapkan untuk mereduksi noise dan memperjelas

bentuk objek, sebelum hasil akhir divisualisasikan dalam bentuk *bounding box* sebagai penanda keberhasilan deteksi. Rangkaian proses ini bertujuan untuk memastikan objek target dapat terisolasi secara akurat dari latar belakang, sehingga sistem mampu menghasilkan deteksi yang stabil dan presisi.

Implementasi sistem deteksi objek pada penelitian ini telah sesuai dengan pendekatan standar pengolahan citra digital menggunakan pustaka OpenCV, khususnya pada domain *color-based object detection*. Penggunaan ruang warna HSV (Hue, Saturation, Value) merupakan praktik umum dalam OpenCV karena komponen *Hue* memungkinkan pemisahan warna objek secara lebih konsisten dibandingkan ruang warna RGB yang sensitif terhadap perubahan intensitas cahaya. Hal ini sangat relevan dalam konteks UAV, di mana kondisi pencahayaan dapat berubah secara dinamis.

Proses *thresholding* pada kanal HSV selaras dengan fungsi OpenCV seperti `inRange()`, yang digunakan untuk menghasilkan citra biner berdasarkan rentang warna tertentu. Visualisasi hasil deteksi menggunakan *bounding box* sesuai dengan metode *contour detection* pada OpenCV, di mana fungsi `findContours()` dan `boundingRect()` digunakan untuk menandai lokasi objek target. Secara keseluruhan, alur pemrosesan yang diterapkan telah mencerminkan pipeline pengolahan citra OpenCV yang sistematis dan efektif untuk kebutuhan deteksi objek berbasis warna secara real-time.

H. Penjelasan screenshot status unggah skrip di Gitlab/Github hingga proyek final



Gambar di atas menampilkan dokumentasi aktivitas pengembangan perangkat lunak menggunakan sistem kontrol versi (*Version Control System*) pada platform GitHub. Tangkapan layar tersebut merekam fase finalisasi proyek yang berlangsung secara intensif pada tanggal 19 hingga 20 Januari 2026. Terlihat adanya kolaborasi multipihak antar-kontributor (*OurinEdnes, pildaa11, Piccel*) dalam melakukan integrasi modul kode. Aktivitas meliputi pengunggahan skrip pembaruan (*push*), penambahan berkas final (*add files*), serta penggabungan cabang pengembangan (*merge branch*) untuk memastikan seluruh komponen sistem A.R.C.S. terintegrasi dengan baik sebelum peluncuran versi final.

I. Analisis Pengerjaan Proyek

Analisis Waktu

Proyek dikerjakan secara bertahap mulai dari perancangan konsep, implementasi algoritma pengolahan citra, hingga pengujian sistem. Waktu pengerjaan relatif sesuai dengan jadwal yang direncanakan meskipun terdapat penyesuaian pada tahap kalibrasi warna.

Ketercapaian Spesifikasi. Sebagian besar spesifikasi aplikasi berhasil dicapai, termasuk deteksi objek berbasis warna, pemrosesan real-time, serta visualisasi bounding box. Sistem telah berjalan sesuai tujuan awal proyek.

Biaya yang Dibutuhkan

Biaya pengembangan relatif rendah karena menggunakan perangkat lunak open-source (Java dan OpenCV) dan pengujian dilakukan menggunakan kamera ponsel sebagai simulasi kamera drone.

Kendala utama yang dihadapi adalah:

- Sensitivitas sistem terhadap pencahayaan
- Jarak kamera terhadap objek
- Noise visual dari lingkungan sekitar

Tantangan dan Pengembangan Masa Depan

Pengembangan selanjutnya dapat dilakukan dengan:

- Penyesuaian threshold warna yang adaptif
- Integrasi dengan drone UAV secara langsung

- Penggunaan metode machine learning untuk meningkatkan akurasi deteksi
- Penambahan fitur pelacakan objek otomatis